STAT 508 - HW4

Ana Camargo

2023-04-08

Chapter 8

```
library(ISLR2)
```

Problem 5

Suppose we produce ten bootstrapped samples from a data set containing red and green classes. We then apply a classification tree to each bootstrapped sample and, for a specific value of X, produce 10 estimates of P(Class is Red|X): 0.1, 0.15, 0.2, 0.2, 0.55, 0.6, 0.6, 0.65, 0.7, and 0.75

- Majority vote: Probabilities > 0.5 are classified as "red", otherwise it is classified as green. So, we have 4 samples classified as "green" and 6 as "red". Therefore, majority voting in this case is "RED" classification (as we have more samples classified as red than green).
- Average probability: If the average probability is > 0.5 then these are classified as "red", otherwise it is as "green". In this case, the average probability among all these 10 is 0.45 (sum of all probabilities/10). Therefore, the classification is "GREEN" based on this approach.

Problem 8

In the lab, a classification tree was applied to the Carseats data set after converting Sales into a qualitative response variable. Now we will seek to predict Sales using regression trees and related approaches, treating the response as a quantitative variable.

(a) Split the data set into a training set and a test set.

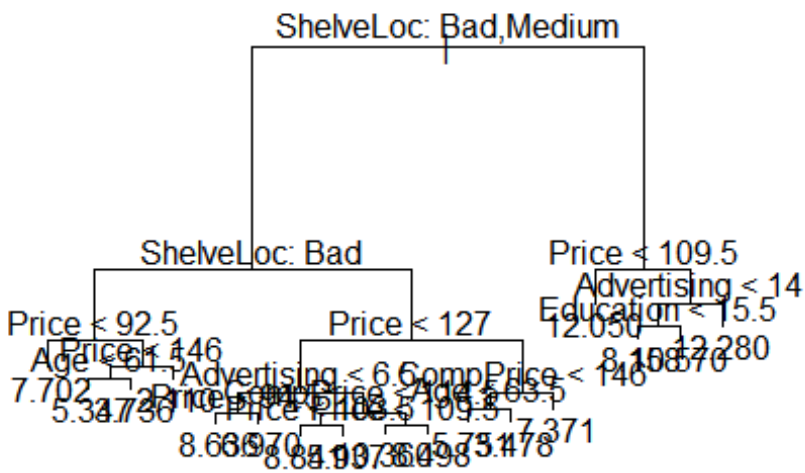
```
data("Carseats")
set.seed(3)
tr1 <- sample(1:nrow(Carseats),nrow(Carseats)*2/3)
te1 <- -tr1
Carseats_tr <- Carseats[tr1,]
Carseats_te <- Carseats[te1,]
```

(b) Fit a regression tree to the training set. Plot the tree, and interpret the results. What test MSE do you obtain?

```
library(tree)

tree_car <- tree(Sales ~ . , data=Carseats_tr)
summary(tree_car)
```

```
##
## Regression tree:
## tree(formula = Sales ~ ., data = Carseats_tr)
## Variables actually used in tree construction:
## [1] "ShelveLoc"   "Price"       "Age"         "Advertising" "CompPrice"
## [6] "Education"
## Number of terminal nodes:  17
## Residual mean deviance:  2.348 = 584.7 / 249
## Distribution of residuals:
##     Min.  1st Qu.   Median     Mean  3rd Qu.     Max.
## -4.23200 -0.98170 -0.02899  0.00000  0.84450  4.72400
```

```
plot(tree_car)
text(tree_car , pretty = 0)
```

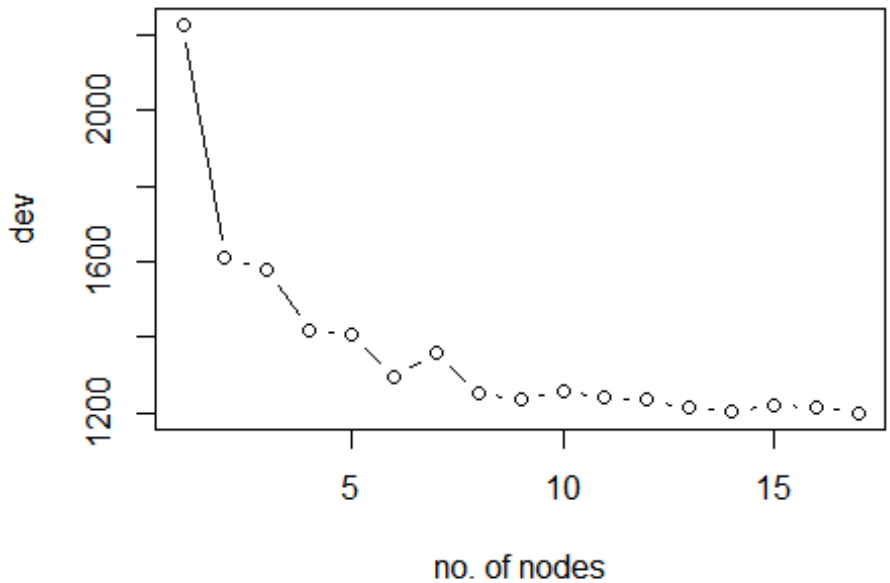



```
tree_pred <- predict(tree_car, Carseats_te)
test_MSE <- mean((Carseats_te$Sales - tree_pred)^2)
test_MSE

## [1] 4.307278
```

The resulting tree has 17 nodes with a mean error of 2.35. From the corresponding plot we can see that the most relevant predictor seems to be the “shelve location”, as this one is the root of the tree. We still need to check how this regression tree performs well on new data. When we use this tree to predict values using the testing set, the corresponding test MSE is still small, 4.3.

(c) Use cross-validation in order to determine the optimal level of tree complexity. Does pruning the tree improve the test MSE?

```
set.seed(3)
cvtree_car <- cv.tree(tree_car)
plot(cvtree_car$size , cvtree_car$dev, type = "b", xlab="no. of nodes",
ylab="dev")
```




```
min(cvtree_car$dev)

## [1] 1199.51
```

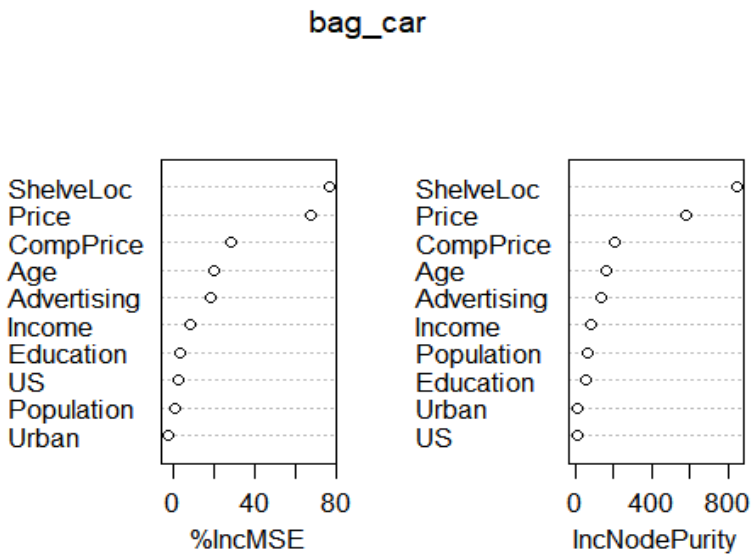
The minimum dev (cross-validation errors) is 1199, which corresponds to the tree with 17 nodes (the original tree). So, pruning the tree does not improve the test MSE as having less nodes would give us higher test MSE. We can fit a prunned tree, with a lower number of nodes, just to check this previous statement. Let us do with 12 nodes.

```
pr_tree <- prune.tree(tree_car, best=12)
prtree_pred <- predict(pr_tree, Carseats_te)
prtest_MSE <- mean((Carseats_te$Sales - prtree_pred)^2)
prtest_MSE

## [1] 5.161296
```

As expected, we get higher test MSE for lower number of terminal nodes. In this case, with 12 nodes we get a test MSE of 5.16 (which is higher than the previous tree with 17 nodes.)

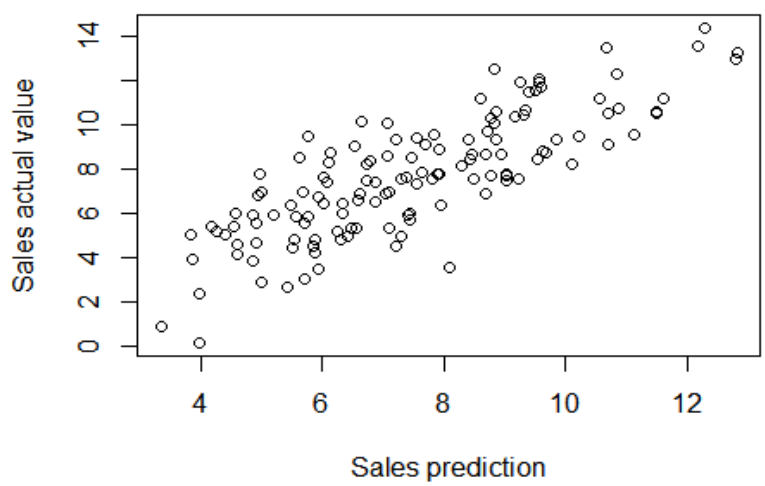
(d) Use the bagging approach in order to analyze this data. What test MSE do you obtain? Use the importance() function to determine which variables are most important.

```
library(randomForest)set.seed(3)
bag_car <- randomForest(Sales~. , data=Carseats_tr, mtry = dim(Carseats)[2]-1
, importance=TRUE)
bag_car

##
## Call:
##  randomForest(formula = Sales ~ ., data = Carseats_tr, mtry =
dim(Carseats)[2] -      1, importance = TRUE)
##                Type of random forest: regression
##                      Number of trees: 500
## No. of variables tried at each split: 10
##
##           Mean of squared residuals: 2.453172
##                     % Var explained: 70.27

importance(bag_car)

##                %IncMSE IncNodePurity
## CompPrice   28.3791758    209.195934
## Income       8.5122552     86.844655
## Advertising 19.0530167    133.522924
## Population   0.8757065     63.134509
## Price       67.7238357    576.629329
## ShelveLoc   77.2761412    847.515525
## Age         20.1743261    166.985605
## Education    3.7751516     55.365616
## Urban       -2.5053898      9.352321
## US           2.6296836      9.233048

varImpPlot(bag_car)
```



Including all 10 predictors for splitting of the data, this indicates the model to do bagging. The train MSE is 2.45. The variables of most importance for the bagging approach are shelve location and price. Let us now check the bagging performance on the test set.

```
bag_pred <- predict(bag_car, Carseats_te)
plot(bag_pred , Carseats_te$Sales, xlab="Sales prediction", ylab="Sales
actual value")
```

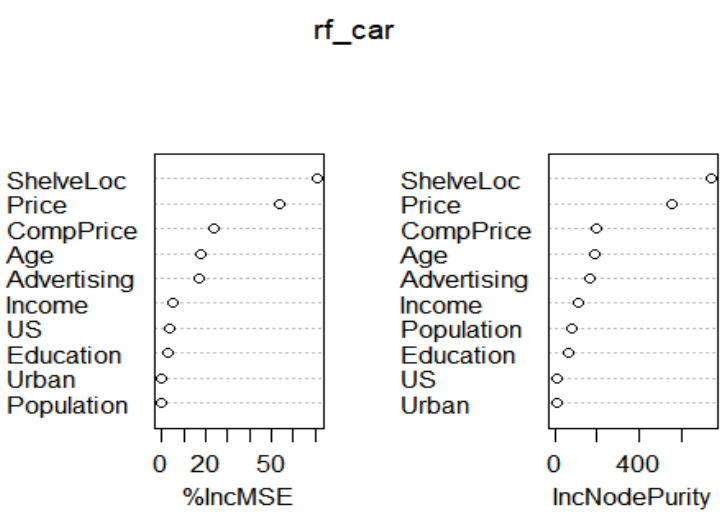



```
bagtest_MSE <- mean((Carseats_te$Sales - bag_pred)^2)
bagtest_MSE

## [1] 2.552804
```

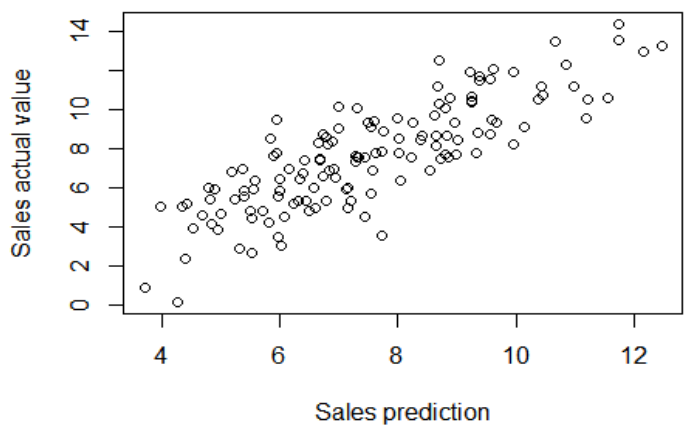
The test MSE obtained using the bagging approach is less than the one obtained with the pruned tree, 2.55 which is approx. half of the first test error.

(e) Use random forests to analyze this data. What test MSE do you obtain? Use the importance() function to determine which variables are most important. Describe the effect of m, the number of variables considered.

```
set.seed(3)
rf_car <- randomForest(Sales~. , data=Carseats_tr, mtry = 6 ,
importance=TRUE)
rf_car

##
## Call:
##  randomForest(formula = Sales ~ ., data = Carseats_tr, mtry = 6,
importance = TRUE)
##                Type of random forest: regression
##                      Number of trees: 500
## No. of variables tried at each split: 6
##
##           Mean of squared residuals: 2.507276
##                     % Var explained: 69.62

importance(rf_car)

##                %IncMSE IncNodePurity
## CompPrice   23.6842513     196.65417
## Income       4.8447957     109.80398
## Advertising 16.5360454     168.05676
## Population  -0.7146618      82.71432
## Price       53.4114930     552.26133
## ShelveLoc   71.2626258     739.38576
## Age         17.4231210     188.32664
## Education    2.2849368      63.06559
## Urban       -0.7129419      11.91378
## US           3.2082949      14.29060

varImpPlot(rf_car)
```



```
rf_pred <- predict(rf_car, Carseats_te)
plot(rf_pred , Carseats_te$Sales, xlab="Sales prediction", ylab="Sales actual
value")
```




```
rftest_MSE <- mean((Carseats_te$Sales - rf_pred)^2)
rftest_MSE

## [1] 2.404951
```

When using random forest, we get a lower test MSE than the one from bagging and the prunned tree. The test MSE in this case is 2.4

Problem 9

This problem involves the OJ data set which is part of the ISLR2 package.

```
data("OJ")
```

(a) Create a training set containing a random sample of 800 observations, and a test set containing the remaining observations.

```
set.seed(9)
tr9 <- sample(1:nrow(OJ),800)
oj_tr <- OJ[tr9,]
oj_te <- OJ[-tr9,]
```

(b) Fit a tree to the training data, with Purchase as the response and the other variables as predictors. Use the summary() function to produce summary statistics about the tree, and describe the results obtained. What is the training error rate? How many terminal nodes does the tree have?

```
tree_oj <- tree(Purchase ~ . , data=oj_tr)
summary(tree_oj)
```

```
##
## Classification tree:
## tree(formula = Purchase ~ ., data = oj_tr)
## Variables actually used in tree construction:
## [1] "LoyalCH"       "PriceDiff"     "ListPriceDiff"
## Number of terminal nodes:  8
## Residual mean deviance:  0.7319 = 579.7 / 792
## Misclassification error rate: 0.1512 = 121 / 800
```

This is a classification tree, as the response variable is qualitative (binomial classification). The training error of the tree is 15.1% which is the proportion of misclassified observations. The tree has 8 terminal nodes.

(c) Type in the name of the tree object in order to get a detailed text output. Pick one of the terminal nodes, and interpret the information displayed.

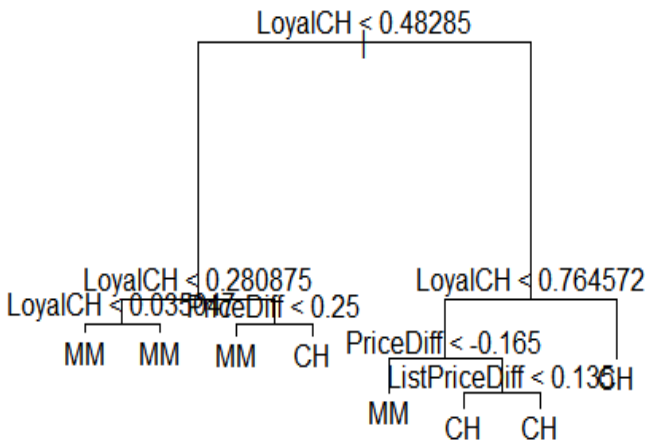
```
tree_oj
```

```
## node), split, n, deviance, yval, (yprob)
##       * denotes terminal node
##
##  1) root 800 1053.000 CH ( 0.63125 0.36875 )
##    2) LoyalCH < 0.48285 288  321.700 MM ( 0.24653 0.75347 )
##      4) LoyalCH < 0.280875 159  120.300 MM ( 0.12579 0.87421 )
##        8) LoyalCH < 0.035047 53    9.922 MM ( 0.01887 0.98113 ) *
##        9) LoyalCH > 0.035047 106   99.690 MM ( 0.17925 0.82075 ) *
##      5) LoyalCH > 0.280875 129  173.100 MM ( 0.39535 0.60465 )
##       10) PriceDiff < 0.25 84   98.620 MM ( 0.27381 0.72619 ) *
##       11) PriceDiff > 0.25 45   59.670 CH ( 0.62222 0.37778 ) *
##    3) LoyalCH > 0.48285 512  437.000 CH ( 0.84766 0.15234 )
##      6) LoyalCH < 0.764572 252  297.800 CH ( 0.72222 0.27778 )
##       12) PriceDiff < -0.165 39   46.400 MM ( 0.28205 0.71795 ) *
##       13) PriceDiff > -0.165 213  211.500 CH ( 0.80282 0.19718 )
##         26) ListPriceDiff < 0.135 36   49.800 CH ( 0.52778 0.47222 ) *
##         27) ListPriceDiff > 0.135 177  144.200 CH ( 0.85876 0.14124 ) *
##      7) LoyalCH > 0.764572 260   71.450 CH ( 0.96923 0.03077 ) *
```

For the terminal node 8, we have the split based on the variable “LoyalCH” taking values less than 0.03, where the leaf has 53 observations and a deviance of 9.922. The classification prediction for this branch is “MM”, as the proportion of values taking this classification is higher than “CH” classification. In this terminal node, 0.01887 of the observations are classified as “CH” and 0.98113 are “MM”.

(d) Create a plot of the tree, and interpret the results.

```
plot(tree_oj)
text(tree_oj , pretty = 0)
```



(e) Predict the response on the test data, and produce a confusion matrix comparing the test labels to the predicted test labels. What is the test error rate?

```
treepred_oj <- predict(tree_oj, oj_te, type="class")
table(treepred_oj,oj_te$Purchase)

##
## treepred_oj  CH  MM
##          CH 132  37
##          MM  16  85

mean(treepred_oj != oj_te$Purchase)

## [1] 0.1962963
```

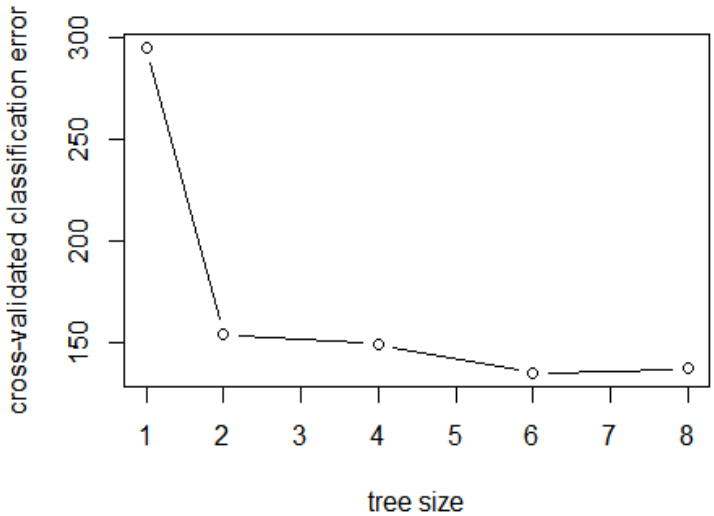
The test error rate, misclassification error, is 19.6% based on the confusion matrix results.

(f) Apply the cv.tree() function to the training set in order to determine the optimal tree size.

```
set.seed(9)
cvtree_oj <- cv.tree(tree_oj, FUN = prune.misclass)
```

(g) Produce a plot with tree size on the x-axis and cross-validated classification error rate on the y-axis.

```
plot(cvtree_oj$size , cvtree_oj$dev, type = "b", xlab="tree size",
ylab="cross-validated classification error")
```



(h) Which tree size corresponds to the lowest cross-validated classification error rate?

The tree with the lowest cross-validated classification error rate is the one with 6 nodes.

(i) Produce a pruned tree corresponding to the optimal tree size obtained using cross-validation. If cross-validation does not lead to selection of a pruned tree, then create a pruned tree with five terminal nodes.

```
pr_oj_tree <- prune.tree(tree_oj, best=6)
```

(j) Compare the training error rates between the pruned and unpruned trees. Which is higher?

```
summary(pr_oj_tree)

##
## Classification tree:
## snip.tree(tree = tree_oj, nodes = 4:5)
## Variables actually used in tree construction:
## [1] "LoyalCH"       "PriceDiff"     "ListPriceDiff"
## Number of terminal nodes:  6
## Residual mean deviance:  0.7623 = 605.2 / 794
## Misclassification error rate: 0.165 = 132 / 800
```

The training error on the pruned tree is 16.5%, which is higher than 15.1% training error from the unpruned tree. However, let us check now the test error to compare the perfomance.

(k) Compare the test error rates between the pruned and unpruned trees. Which is higher?

```
pr_oj_tree_pred <- predict(pr_oj_tree, oj_te, type="class")
table(pr_oj_tree_pred,oj_te$Purchase)
```

```
##
## pr_oj_tree_pred  CH  MM
##              CH 124  26
##              MM  24  96
```

```
mean(pr_oj_tree_pred != oj_te$Purchase)
```

```
## [1] 0.1851852
```

The pruned tree with 6 terminal nodes (optimal tree size gotten by cross-validation) results in a lower test error than the unpruned tree. With the pruned tree, we get a test error of 18.5% compared to 19.6%

Problem 10

We now use boosting to predict Salary in the Hitters data set.

```
data("Hitters")
```

(a) Remove the observations for whom the salary information is unknown, and then log-transform the salaries.

```
new_hit <- na.omit(Hitters)
new_hit$Salary <- log(new_hit$Salary)
```

(b) Create a training set consisting of the first 200 observations, and a test set consisting of the remaining observations.

```
hit_tr <- new_hit[1:200, ]
hit_te <- new_hit[201:nrow(new_hit), ]
```

(c) Perform boosting on the training set with 1,000 trees for a range of values of the shrinkage parameter λ. Produce a plot with different shrinkage values on the x-axis and the corresponding training set MSE on the y-axis

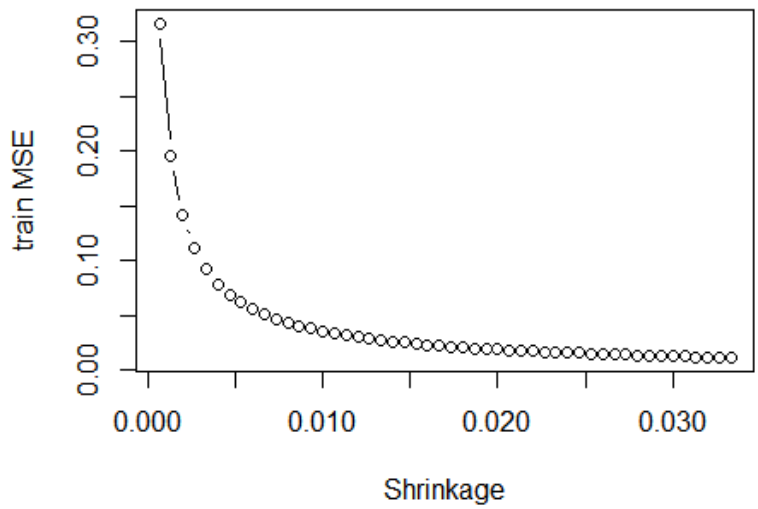
```
library(gbm)
```

```
## Warning: package 'gbm' was built under R version 4.2.3
```

```
## Loaded gbm 2.1.8.1
```

```
t = 50
n = rep(NA,t)
tr_MSE = rep(NA,t)
te_MSE = rep(NA,t)

for (i in 1:t){
  n[i] = i/1500
  set.seed(10)
  boost_hit <- gbm(Salary ~ ., data = hit_tr, distribution = "gaussian",
n.trees = 1000, interaction.depth = 4, shrinkage = n[i])
  tr_MSE[i] <- mean((boost_hit$train.error)^2)
```

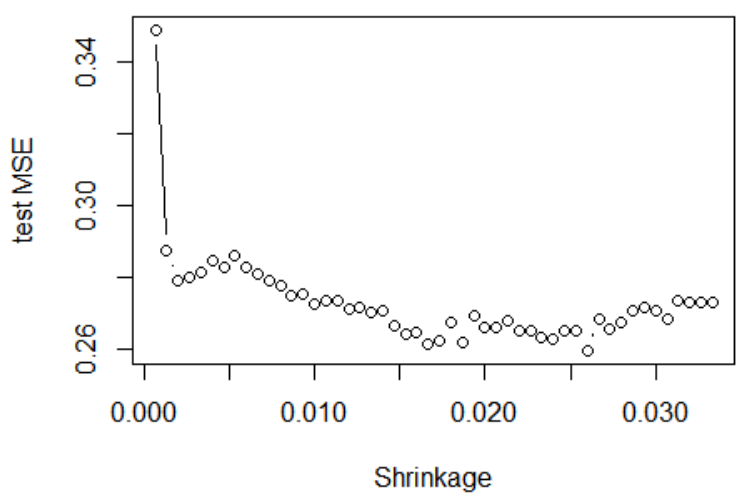
```
  pred_hit <- predict(boost_hit , newdata = hit_te, n.trees = 1000)
  te_MSE[i] <- mean(( pred_hit - hit_te$Salary)^2)
}
df_mse <- data.frame(shrinkage = n, train_MSE = tr_MSE, test_MSE = te_MSE)
plot(df_mse$shrinkage, df_mse$train_MSE, type="b", xlab="Shrinkage",
ylab="train MSE")
```



As we can see on the plot, the training MSE decreases with higher values of shrinkage, as the model is learning from the previous fitted model in an iterative way.

(d) Produce a plot with different shrinkage values on the x-axis and the corresponding test set MSE on the y-axis.

```
plot(df_mse$shrinkage, df_mse$test_MSE, type="b", xlab="Shrinkage",
ylab="test MSE")
```




```
min(df_mse$test_MSE)

## [1] 0.2593592
```

```
z <- df_mse$shrinkage[which(df_mse$test_MSE == min(df_mse$test_MSE))]
z

## [1] 0.026
```

The testing MSE for diferente values of shrinkage has a slightly different behaviour than the training error. OVerall, this seems to be decreasing but has some variations on the way. The lowest test error (0.26) is obtained with a shrinkage of 0.026, which will be used to fit the "best" boosting model.

(e) Compare the test MSE of boosting to the test MSE that results from applying two of the regression approaches seen in Chapters 3 and 6.

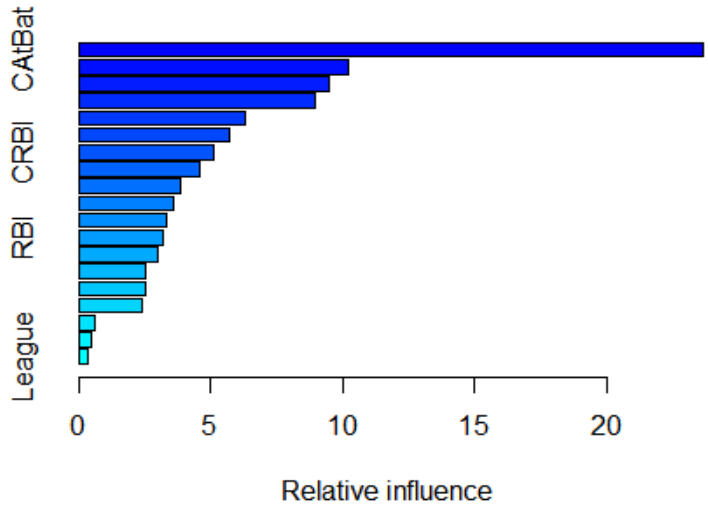
```
# boosting model using best shrinkage
boost_hit <- gbm(Salary ~ ., data = hit_tr, distribution = "gaussian",
n.trees = 1000, interaction.depth = 4, shrinkage = z)
# linear regression
lm_hit <-  lm(data=hit_tr, Salary ~ .)
# lasso regression with lambda by cross-validation
library(glmnet)

## Loading required package: Matrix

## Loaded glmnet 4.1-6

M <- model.matrix(Salary ~ . , data=hit_tr)[,-1]
M2 <- model.matrix(Salary ~ . , data=hit_te)[,-1]
cv_lasso <- cv.glmnet( M , hit_tr$Salary , alpha = 1)
lasso_hit <- glmnet(M , hit_tr$Salary , alpha = 1, lambda =
cv_lasso$lambda.min)

# predictions for each model
pred_hit_boost <- predict(boost_hit, newdata = hit_te, n.trees = 1000)
pred_hit_lm <- predict(lm_hit, newdata = hit_te)
pred_hit_ls <- predict(lasso_hit, newx = M2)

# test errors for each model
test_mse_boost <- mean(( pred_hit_boost - hit_te$Salary)^2)
test_mse_lm <- mean(( pred_hit_lm - hit_te$Salary)^2)
test_mse_ls <- mean(( pred_hit_ls - hit_te$Salary)^2)
df_mod_mse <- data.frame(model = c('boosting','linear reg','lasso reg'),
test_MSE = c(test_mse_boost,test_mse_lm,test_mse_ls))
df_mod_mse

##        model  test_MSE
## 1   boosting 0.2728775
## 2 linear reg 0.4917959
## 3  lasso reg 0.4717890
```

When we use boosting, fit a linear model, and a lasso regularized model, we get different test errors on the same data set. From the above table we can see that the lowest test error

is obtained when using Boosting (0.27) and the highest is obtained with the linear model (0.49).

(f) Which variables appear to be the most important predictors in the boosted model?

```
summary(boost_hit)
```




```
##                  var    rel.inf
## CAtBat       CAtBat 23.6776632
## CHits         CHits 10.2473989
## CWalks       CWalks  9.5052824
## CRuns         CRuns  8.9651828
## Walks         Walks  6.2931739
## PutOuts     PutOuts  5.7283607
## CRBI           CRBI  5.0857338
## Years         Years  4.5850779
## CHmRun       CHmRun  3.8590472
## Assists     Assists  3.5874848
## AtBat         AtBat  3.3406773
## RBI             RBI  3.1749199
## HmRun         HmRun  2.9887659
## Hits           Hits  2.5610933
## Runs           Runs  2.5439145
## Errors       Errors  2.3858851
## NewLeague NewLeague  0.6209818
## Division   Division  0.5093154
## League       League  0.3400413
```

The most important predictors in the boosted model seems to be (by far) CAtBat with the highest influence, followed by CHits and CWalks.

(g) Now apply bagging to the training set. What is the test set MSE for this approach?

```
bag_hit <- randomForest(Salary~. , data=hit_tr, mtry = dim(hit_te)[2]-1 ,
importance=TRUE)
pred_bag_hit <- predict(bag_hit, hit_te)
```

```
hit_bag_MSE <- mean((hit_te$Salary - pred_bag_hit)^2)
hit_bag_MSE

## [1] 0.2349253
```

When using bagging in the training set, we get a test MSE of 0.23, which is a little smaller than the test MSE obtained with boosting. So, overall methods, this seems to be the best approach in this case.

Problem 11

This question uses the Caravan data set.

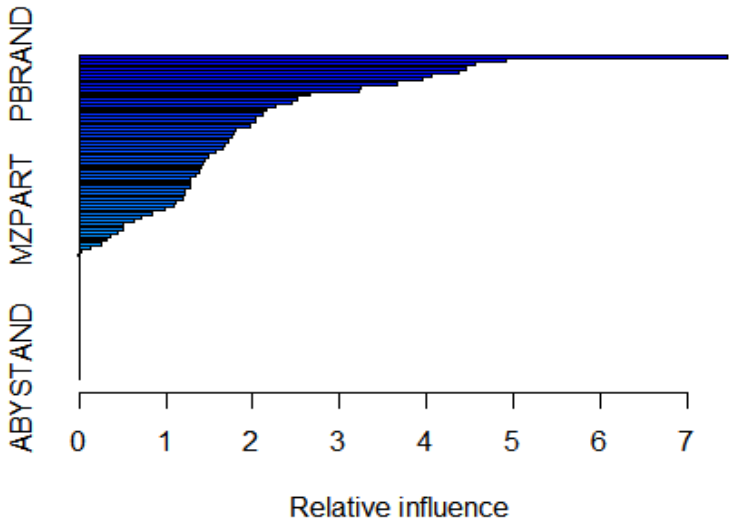
```
data("Caravan")
Caravan$Purchase <- as.character(Caravan$Purchase)
Caravan$Purchase[Caravan$Purchase == 'Yes'] <- 1
Caravan$Purchase[Caravan$Purchase == 'No'] <- 0
```

(a) Create a training set consisting of the first 1,000 observations, and a test set consisting of the remaining observations.

```
carv_tr <- Caravan[1:1000,]
carv_te <- Caravan[1001:nrow(Caravan),]
```

(b) Fit a boosting model to the training set with Purchase as the response and the other variables as predictors. Use 1,000 trees, and a shrinkage value of 0.01. Which predictors appear to be the most important?

```
set.seed(11)
boost_carv <- gbm(Purchase ~ ., data = carv_tr, distribution = "bernoulli",
n.trees = 1000, interaction.depth = 4 ,shrinkage = 0.01)

## Warning in gbm.fit(x = x, y = y, offset = offset, distribution =
distribution,
## : variable 50: PVRAAUT has no variation.

## Warning in gbm.fit(x = x, y = y, offset = offset, distribution =
distribution,
## : variable 71: AVRAAUT has no variation.

summary(boost_carv)
```




```
##                var     rel.inf
## PPERSAUT PPERSAUT 7.472017283
## MGODGE     MGODGE 4.920268524
## PBRAND     PBRAND 4.562337369
## MKOOPKLA MKOOPKLA 4.455474976
## MOPLHOOG MOPLHOOG 4.380527334
## MOSTYPE   MOSTYPE 4.051368353
## MINK3045 MINK3045 3.950103516
## MGODPR     MGODPR 3.672167279
## MBERMIDD MBERMIDD 3.243664884
## MAUT2       MAUT2 3.234222424
## MBERARBG MBERARBG 2.649691655
## MSKB1       MSKB1 2.519943184
## MSKC         MSKC 2.458311336
## MOPLMIDD MOPLMIDD 2.265387824
## MSKA         MSKA 2.160331133
## PWAPART   PWAPART 2.124682559
## MBERARBO MBERARBO 2.033834610
## MINKM30   MINKM30 2.025470388
## MFWEKIND MFWEKIND 1.973111208
## MRELOV     MRELOV 1.811226920
## MBERHOOG MBERHOOG 1.779508314
## MAUT1       MAUT1 1.761319430
## MRELSA     MRELSA 1.720751942
## MRELGE     MRELGE 1.667203409
## MGODOV     MGODOV 1.649518856
## MINK7512 MINK7512 1.580946746
## MINKGEM   MINKGEM 1.495104835
## MFALLEEN MFALLEEN 1.445414183
## MSKB2       MSKB2 1.417953954
## MZFONDS   MZFONDS 1.401154197
## MAUT0       MAUT0 1.379730933
## MGODRK     MGODRK 1.346956354
## MINK4575 MINK4575 1.289878285
```

```
## MFGEKIND MFGEKIND 1.287102544
## ABRAND     ABRAND 1.269977976
## MGEMLEEF MGEMLEEF 1.210600254
## MHHUUR     MHHUUR 1.207518555
## MGEMOMV   MGEMOMV 1.188485795
## MSKD         MSKD 1.120963005
## MHKOOP     MHKOOP 1.097480654
## MZPART     MZPART 0.981066976
## APERSAUT APERSAUT 0.836917534
## MOSHOOFD MOSHOOFD 0.716215569
## MOPLLAAG MOPLLAAG 0.627775856
## MBERZELF MBERZELF 0.509784887
## PMOTSCO   PMOTSCO 0.498942450
## MBERBOER MBERBOER 0.440908428
## PLEVEN     PLEVEN 0.365185344
## PBYSTAND PBYSTAND 0.310737284
## MINK123M MINK123M 0.252407865
## MAANTHUI MAANTHUI 0.132457819
## ALEVEN     ALEVEN 0.025485709
## PFIETS     PFIETS 0.013999110
## PAANHANG PAANHANG 0.006402191
## PWABEDR   PWABEDR 0.000000000
## PWALAND   PWALAND 0.000000000
## PBESAUT   PBESAUT 0.000000000
## PVRAAUT   PVRAAUT 0.000000000
## PTRACTOR PTRACTOR 0.000000000
## PWERKT     PWERKT 0.000000000
## PBROM       PBROM 0.000000000
## PPERSONG PPERSONG 0.000000000
## PGEZONG   PGEZONG 0.000000000
## PWAOREG   PWAOREG 0.000000000
## PZEILPL   PZEILPL 0.000000000
## PPLEZIER PPLEZIER 0.000000000
## PINBOED   PINBOED 0.000000000
## AWAPART   AWAPART 0.000000000
## AWABEDR   AWABEDR 0.000000000
## AWALAND   AWALAND 0.000000000
## ABESAUT   ABESAUT 0.000000000
## AMOTSCO   AMOTSCO 0.000000000
## AVRAAUT   AVRAAUT 0.000000000
## AAANHANG AAANHANG 0.000000000
## ATRACTOR ATRACTOR 0.000000000
## AWERKT     AWERKT 0.000000000
## ABROM       ABROM 0.000000000
## APERSONG APERSONG 0.000000000
## AGEZONG   AGEZONG 0.000000000
## AWAOREG   AWAOREG 0.000000000
## AZEILPL   AZEILPL 0.000000000
## APLEZIER APLEZIER 0.000000000
## AFIETS     AFIETS 0.000000000
```

```
## AINBOED   AINBOED 0.000000000
## ABYSTAND ABYSTAND 0.000000000
```

The most important predictor appears to be PPERSAUT, followed by MGODGE, PBRAND, MKOOPKLA and MOPLHOOG with which we get the top five most influential predictors in this boosting.

(c) Use the boosting model to predict the response on the test data. Predict that a person will make a purchase if the estimated probability of purchase is greater than 20 %. Form a confusion matrix. What fraction of the people predicted to make a purchase do in fact make one?

```
pred_carv_boost <- predict(boost_carv, newdata = carv_te, n.trees = 1000,
type="response")
pred_carv_boost[pred_carv_boost > 0.20] <- 1
pred_carv_boost[pred_carv_boost <= 0.20] <- 0
table(pred_carv_boost, carv_te$Purchase)

##
## pred_carv_boost    0    1
##               0 4346  254
##               1  187   35

35/(187+35)

## [1] 0.1576577
```

When testing the boosting model in the test set, we get a confusion matrix that suggests a 91% of accuracy. However, when we check the fraction of the people predicted to make a purchase that in fact make one, we get 0.16